

What's New in ClaudeFast v5.3

Cross-Device Library + Smarter Sub-Agent Defaults

May 2026 · Upgrading from v5.2

v5.3 is a **refinement release** on top of the v5.2 Library System launch. The headline changes: **the central library now works cleanly across multiple machines**, **sub-agents default to the right model tier** instead of burning Opus tokens on routine work, and **the LibraryHook architecture was simplified to a single synchronous Stop hook**. **ContextRecoveryHook** fixes eliminate duplicate backup files, and `cf-codestats` is now live on PyPI.

2

MODEL TIERS

1

HOOK (WAS 3)

3

BACKUP BUGS FIXED

PyPI

CF-CODESTATS LIVE

Cross-Device Library Sync

In v5.2, the library clone path was baked into each project's manifest. That worked on one machine but broke silently when a second device opened the same project and couldn't find the recorded path. v5.3 moves the library path out of the project manifest and into a per-device registry.

How it works

- New per-device file at `~/.claude/library-paths.json`, keyed by the library's git remote URL (the only stable identifier across machines)
- New command: `node sync.mjs --link` registers your local library clone for this device. Run it once per machine.
- `node sync.mjs --unlink` removes the entry from the registry
- Resolver chain: `CLAUDE_LIBRARY_PATH` env var (if set) → registry lookup by remote → autodetect under common roots (`~/GitHub`, `~/code`, `~/src`, `~/Projects`) → actionable error

Migration

Existing projects auto-migrate on the next sync. A stale `library_path` in your manifest is read once, auto-registered if the path still exists on disk, and stripped on the next manifest write. No customer action required for the common case.

Manifest filename

The library manifest is now named `library.json` (renamed from the legacy `.library-manifest.json`). Sync tooling reads either name, writes the new one, and removes the legacy file on first sync. No action needed.

Smarter Sub-Agent Model Defaults

All 18 specialist agents previously defaulted to `model: opus`. Routine specialist work was burning Opus tokens for tasks Sonnet handles cleanly. v5.3 splits agents into two tiers based on the cost-of-being-wrong of their typical output.

Why Sonnet for execution

In the standard `/team-plan + /build` pipeline, the high-leverage stages stay on Opus and the middle stage moves to Sonnet:

- **Planning — Opus.** Master Orchestrator decomposes the request, assigns specialists, and writes the plan file. Deeper reasoning here changes the outcome of the whole run.
- **Execution — Sonnet.** The specialists implement against the plan. This stage is where the bulk of token spend happens, and it's also where Sonnet matches Opus most closely.
- **Review & validation — Opus.** frontend-specialist and debugger-detective stay on Opus to catch subtle quality and correctness issues before the work ships.

In practice the execution layer produces about **99.5% identical code quality** on Sonnet versus Opus, at **significantly higher speed, considerably lower token cost**, and noticeably better throughput per session. You barely see a difference in output, but you do see a real difference in efficiency. Keeping Opus on the bookends (plan + review) and Sonnet in the middle is the sweet spot.

Tier	Agents	Why
Opus (5)	content-writer, growth-engineer, frontend-specialist, master-orchestrator, debugger-detective	Orchestration, creative, and high-stakes debugging where deeper reasoning materially changes the outcome
Sonnet (13)	backend-engineer, code-simplifier, deep-researcher, flutter-expert, ios-expert, n8n-builder, performance-optimizer, quality-engineer, security-auditor, seo-specialist, session-librarian, supabase-specialist, visual-explainer	Focused, well-bounded specialist work where Sonnet's quality is more than sufficient and the throughput / cost win is real

CLAUDE.md guidance

The blanket Opus mandate in CLAUDE.md was replaced with override-up-to-Opus guidance:

"Sub-agent files have default model definitions in their YAML frontmatter. Feel free to override up to Opus when the work is critical, highly important, or highly challenging."

You don't need to think about this for most prompts -- Central AI picks the right tier automatically. When you want to force Opus for a particular task, edit the agent's frontmatter or pass an override at dispatch time.

LibraryHook: Single Stop Hook (was 3 files)

The v5.2 LibraryHook ran on three triggers: PostToolUse caught Claude's tool edits, UserPromptSubmit ran an mtime walk to catch manual IDE edits, and a 180-second detached worker debounced rapid edits into batched pushes. The architecture worked but flashed a console window on Windows every time the worker spawned, and the moving parts made debugging painful.

v5.3 collapses all of this into a single Stop hook:

- Fires once at the end of every Claude turn (the `Stop` event)
- Runs synchronously inside Claude Code -- no detached process
- Walks file mtimes against the manifest, pushes anything that actually changed, exits
- End-to-end push takes ~4-5 seconds when there's something to sync, instant when there isn't

- No more Windows console-window flash

Files removed: `library-mtime-check.mjs` and `library-sync-worker.mjs` . New file: `library-path-resolver.mjs` (powers the cross-device resolver chain above).

ContextRecoveryHook Reliability Fixes

Three compounding bugs in the auto-backup hook were repaired this release. Together they eliminate duplicate backup files, wrong status-line paths, and silent backup failures.

Bug A: Markdown formatter crash on file changes

A reference to `projectDir` from outside its scope crashed the markdown formatter whenever a session had files modified, silently failing the backup. Fixed by hoisting the variable to module scope.

Bug B: Cross-session contamination from global state

The hook's state file was a single global at `~/.claude/claudefast-statusline-state.json` . Two concurrent Claude Code sessions on the same machine saw each other's session ID, reset their backup counters, and re-tagged every backup as `_first` -- creating brand-new numbered files instead of overwriting the existing one. The same mechanism let one project's backup path leak into another project's status line.

Fixed: state and lock files moved to per-project, per-session paths under `<project>/.claude/.claudefast-state-<sessionId>.json` . The compactor lock stays global since it serializes an expensive summarization step.

Bug C: Within-session race on state writes

Two statusline ticks within ~50ms could both read state, modify it, and overwrite each other's update. Fixed with atomic writes: `writeFileSync(tmp); renameSync(tmp, target)` , with PID + timestamp + random tmp filenames so concurrent writers never share a path.

Cleanup sweep

State and lock files older than 7 days are now deleted on the first backup of each process, gated by a per-process flag so it doesn't fire on every statusline tick. Active session files are preserved.

cf-codestats on PyPI

v5.2 shipped the `codestats` skill but the underlying CLI package had only been built locally and never uploaded to PyPI. A customer hit `ERROR: Could not find a version that satisfies the requirement cf-codestats on pip install` . v0.4.0 is now live on PyPI.

```
pip install cf-codestats
```

The `codestats` SKILL.md also gained a GitHub install fallback for restricted or offline environments where PyPI isn't reachable:

```
pip install git+https://github.com/Abdo-El-Mobayad/codestats.git
```

Upgrade Notes

v5.3 is a drop-in replacement for v5.2. Extract the new `.claude` folder and `CLAUDE.md` over your existing installation. Your custom skills, session files, and project-specific configuration are preserved.

Key changes to be aware of:

- **Per-device library setup is required for the central library** -- Run `node sync.mjs --link` from your library directory once per machine. Without it, the LibraryHook can't find the local clone. (Doesn't affect you if you're not using the central library.)
- **Two LibraryHook files were removed** -- `library-mtime-check.mjs` and `library-sync-worker.mjs`. If you're applying v5.3 over an existing v5.2 install, delete those two files. They're inert if left behind (settings.json no longer references them) but they're dead weight.
- **Specialists now default to Sonnet** -- 13 of 18 agents flipped from Opus to Sonnet. Override per task or per agent file when you want deeper reasoning. `CLAUDE.md` replaced the previous blanket Opus mandate with override-up-to-Opus guidance.
- **Manifest renamed to `library.json`** -- Auto-migrates from the legacy `.library-manifest.json` on first sync. No action required.
- **cf-codestats now installs from PyPI** -- `pip install cf-codestats` works on the official index. If PyPI isn't reachable, the `SKILL.md` documents a GitHub fallback.

Backups will overwrite cleanly now. If you've been seeing duplicate numbered backup files in `.claude/backups/` from concurrent sessions, the v5.3 ContextRecoveryHook fixes resolve that. Old duplicate files are safe to delete.